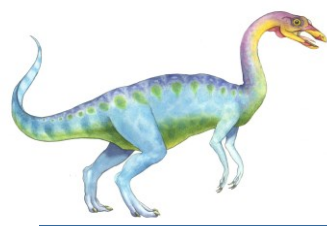




Assignment 3: High-Weight Contribution to Final Grade





Priority Scheduling Algorithm

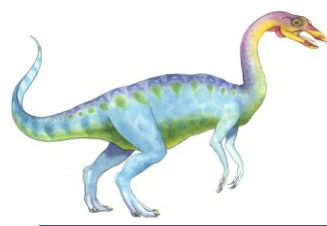
Process	Arrival Time	Burst Time	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	1
P5	9	16	4

Non-Preemptive

P1	P2	P4	P3	P5	
0	11	39	49	51	67

Avg WT : 5.6
Avg TAT : 8.8





Priority Scheduling Algorithm

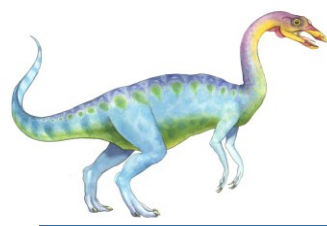
Process	Arrival Time	Priority	Burst Time
P1	0	3	3
P2	1	2	4
P3	2	4	6
P4	3	6	4
P5	5	10	2

Preemptive

P1	P2	P2	P2	P1	P3	P4	P5	
0	1	2	3	5	7	13	17	19

Avg WT : 10
Avg TAT : 6.2
Avg R.T : 5.4 ?





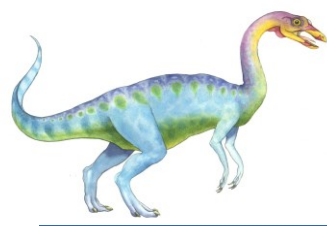
Priority Scheduling Algorithm

Process	Arrival Time	Priority	Burst Time
P1	0	3	8
P2	1	4	2
P3	3	4	4
P4	4	5	1
P5	5	2	6
P6	6	6	5
P7	10	1	1

Preemptive

Avg WT :
Avg TAT :
Avg R.T : ?





Priority Scheduling Algorithm

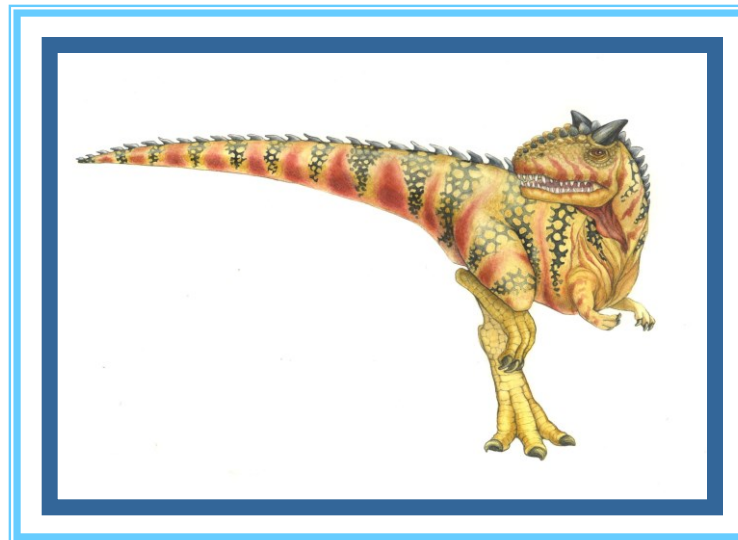
Preemptive

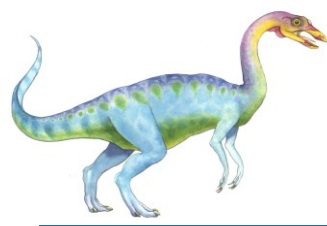
Process	Arrival Time	Priority	Burst Time
P1	0	3	8
P2	1	4	2
P3	3	4	4
P4	4	5	1
P5	5	2	6
P6	6	6	5
P7	10	1	1

P1	P1	P1	P1	P5	P5	P7	P5	P1	P2	P3	P4	P6	
0	1	3	4	5	6	10	11	12	15	17	21	22	27



Process Synchronization





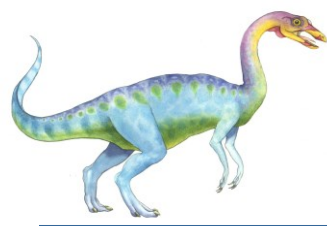
Why Process Synchronization ?

	Process P	Process J
Shared_Variable=10	P1 R1= shared_variable	J1 R2= shared_variable
	P2 R2= R1 + 1	J2 R2= R2 - 1
	Pre-Empt	Pre-Empt
	P3 Shared_variable = R1	J3 Shared_variable = R2

P1, P2, J1,J2,P3, J3 ?

P1, P2, J1,J2,J3, P3 ?





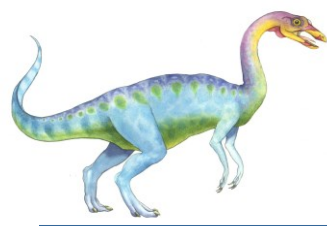
Why Process Synchronization ?

	Process P	Process J
Shared_Variable=10	P1 R1= shared_variable	J1 R2= shared_variable
	P2 R2= R1 + 1	J2 R2= R2 - 1
	Pre-Empt	Pre-Empt
	P3 Shared_variable = R1	J3 Shared_variable = R2

P1, P2, J1,J2,P3, J3 = 9

P1, P2, J1,J2,J3, P3 = 11





Race Condition

- `counter++` could be implemented as

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

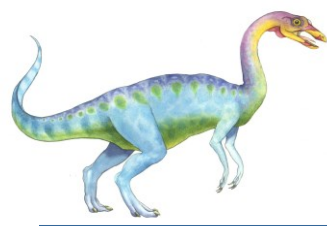
- `counter--` could be implemented as

```
register2 = counter  
register2 = register2 - 1  
count = register2
```

- Consider this execution interleaving with “**count = 5**” initially:

```
S0: producer execute register1 = counter {register1 = 5}  
S1: producer execute register1 = register1 + 1 {register1 = 6}  
S2: consumer execute register2 = counter {register2 = 5}  
S3: consumer execute register2 = register2 - 1 {register2 = 4}  
S4: producer execute counter = register1 {count = 6}  
S5: consumer execute counter = register2 {count = 4}
```



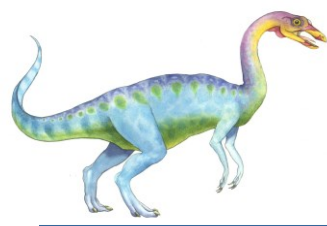


Race Condition

When more than one processes are executing the same code or accessing the same memory or any shared variable in that condition there is a possibility that the output or the value of the shared variable is wrong so for that all the processes doing the race to say that my output is correct this condition known as a race condition. Several processes access and process the manipulations over the same data concurrently, then the outcome depends on the particular order in which the access takes place.

The order of execution of instructions defines the output.





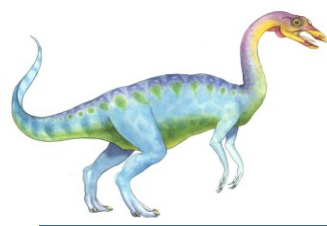
Critical Section

Critical section is a code segment that contains shared variables that's why it need to be synchronized to maintain consistency of data variables. Critical Section can be accessed by only one process at a time.

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (TRUE);
```

Figure 6.1 General structure of a typical process P_i .





Critical Section

Entry Section It is part of the process, which decide the entry of a particular process in the Critical Section, out of many other processes.

Critical Section - It is the part in which only one process is allowed to enter and modify the shared variable. This part of the process ensures that only no other process can access the resource of shared data.

Exit Section - This process allows the other process that are waiting in the Entry Section, to enter into the Critical Sections. It checks that a process that after a process has finished execution in Critical Section can be removed through this Exit Section.

Remainder Section The other parts of the Code other than Entry - Section, Critical Section and Exit Section are known as Remainder Section.

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (TRUE);
```





Critical-Section Problem

Each process has a section of code, called the **critical section**, in which the process access and manipulate **shared variables and files**.

A **race condition** is a situation that may occur inside a critical section.

The problem is to ensure that when one process is executing in its critical section then no other process can execute their critical section

Race conditions in critical sections can be avoided if the critical section is treated as an **atomic instruction**.

This can be done by **process synchronization**.





Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.





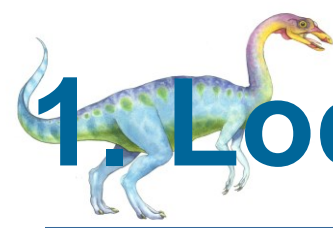
Synchronization Mechanism

Mutual Exclusion

Progress

Bounded Waiting





1. Lock Variable Synchronization Mechanism

Its a software mechanism implemented in user mode, i.e. no support required from the Operating System.

Its a busy waiting solution.

It can be used for more than two processes.

do {

 acquire Lock

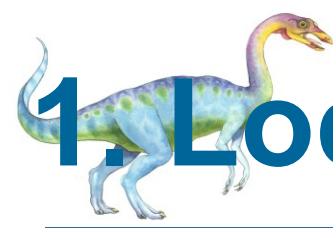
CRITICAL SECTION

 release Lock

 Remainder Section

} while(TRUE)





1. Lock Variable Synchronization Mechanism

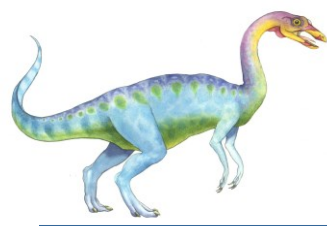
```
while(lock != 0); // While (lock ==1);  
Lock = 1;
```

CRITICAL SECTION

```
Lock = 0;  
Remainder Section
```

M.E
P
B.W
?





2. Test And Set Instruction

```
while(lock != 0); // While (lock ==1);
```

```
Lock = 1;
```

CRITICAL SECTION

```
Lock = 0;
```

Remainder Section

```
while(1) {  
    while (TestAndSet(&lock));  
        critical section  
    lock = false;  
    remainder section  
}
```

```
boolean TestAndSet (*target)  
{  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}
```

It requires the operating system support
It is a busy waiting solution.

M.E
P
B.W
?



Reference: Chapter 6

